SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

CONCEPT MAPPING

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

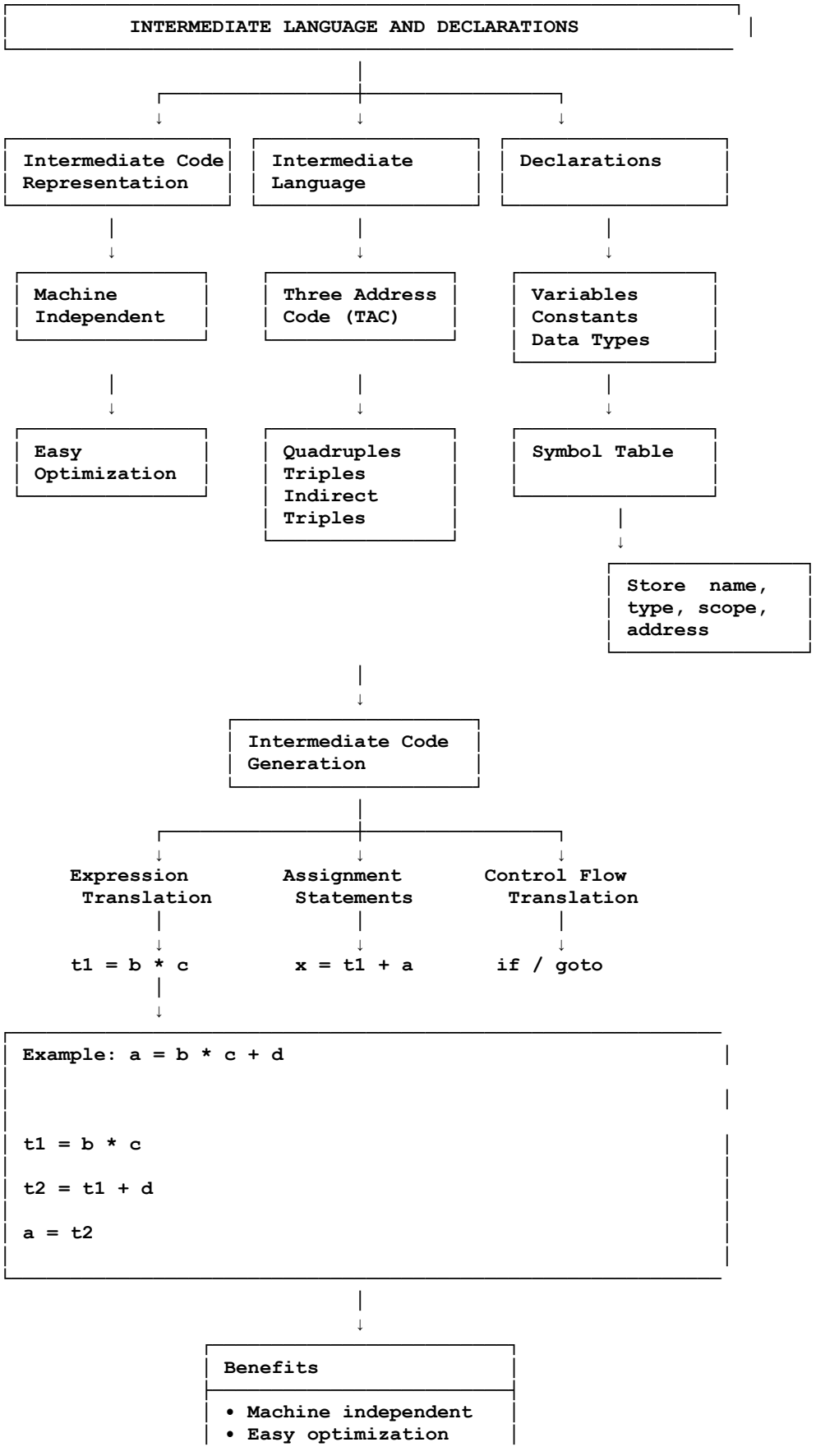
CONCEPT MAPPING

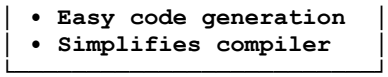
1.Intermediate Language and Declarations
2.Assignment Statements and Boolean Expressions
3.Case Statements and Backpatching

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

1. INTERMEDIATE LANGUAGE AND DECLARATIONS

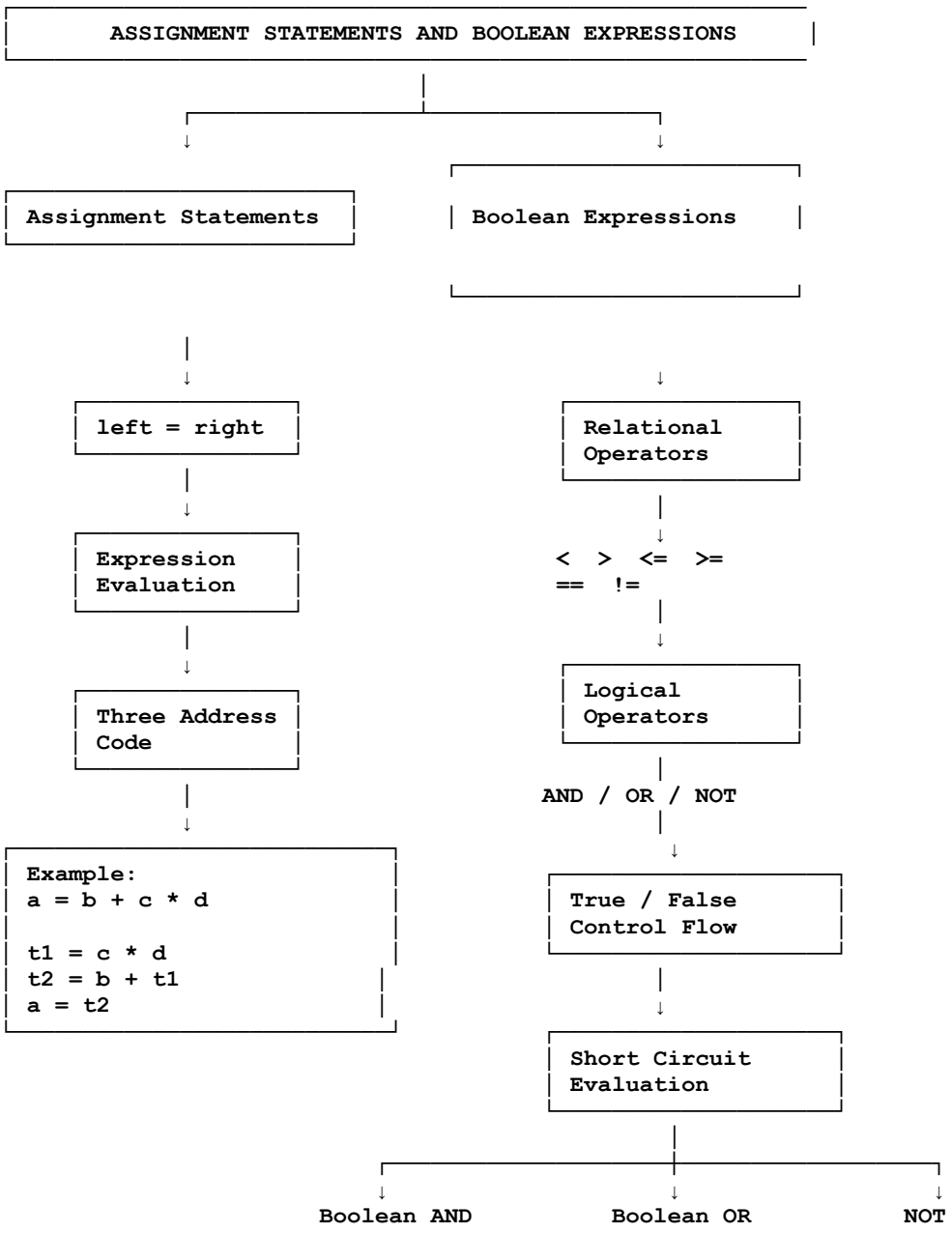


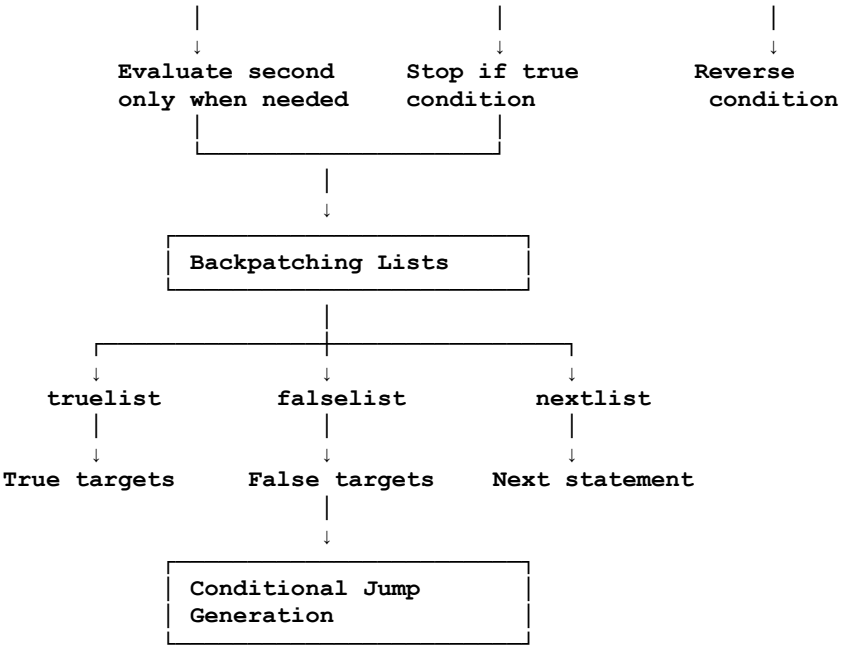


Conclusion:

Intermediate language acts as a bridge between the source program and target machine code. Three-address code and declaration information provide a structured representation that makes optimization and final code generation easier and more efficient.

2. ASSIGNMENT STATEMENTS AND BOOLEAN EXPRESSIONS

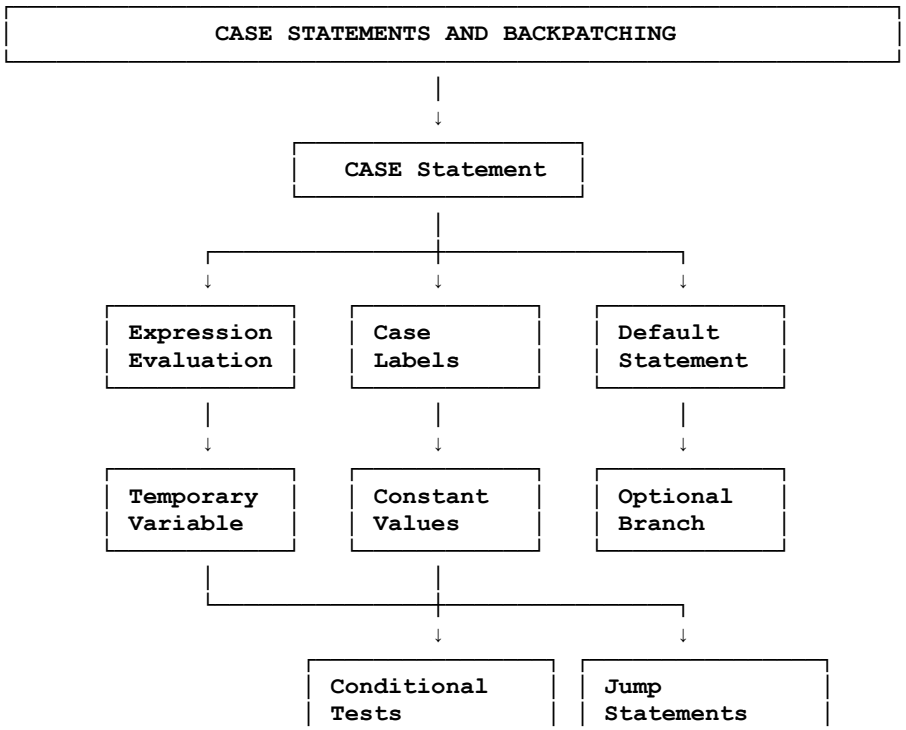


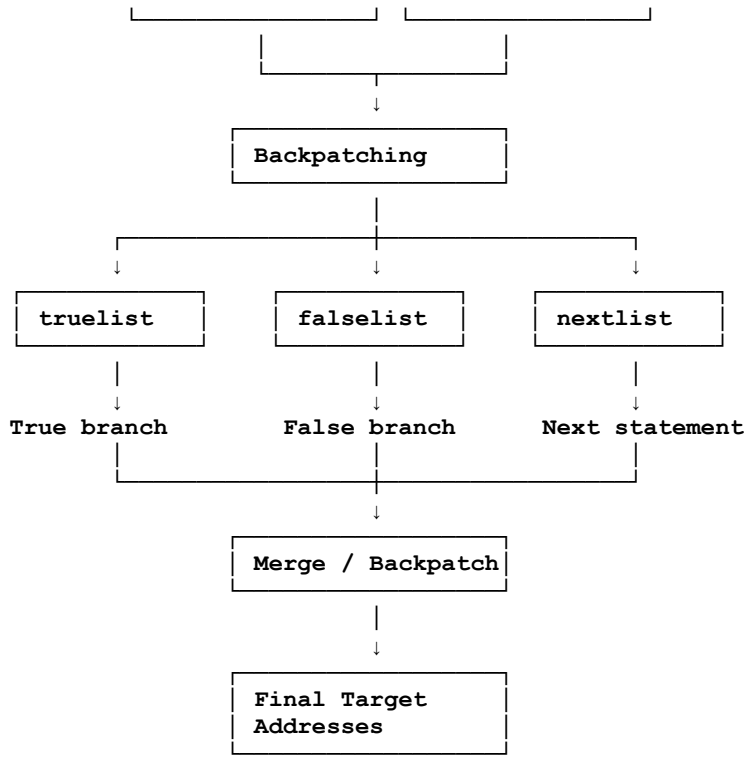


Conclusion:

Assignment statements translate expressions into simple three-address instructions, while Boolean expressions generate conditional jumps. Short-circuit evaluation and backpatching help the compiler efficiently manage control flow and unresolved jump targets.

3. CASE STATEMENTS AND BACKPATCHING





Conclusion:

Case statements create multiple control-flow paths based on the value of an expression. Backpatching resolves incomplete jump targets after the required labels become known, allowing the compiler to generate correct and efficient intermediate code.